

■ Transformers, LoRA, QLoRA & LLM Deployment

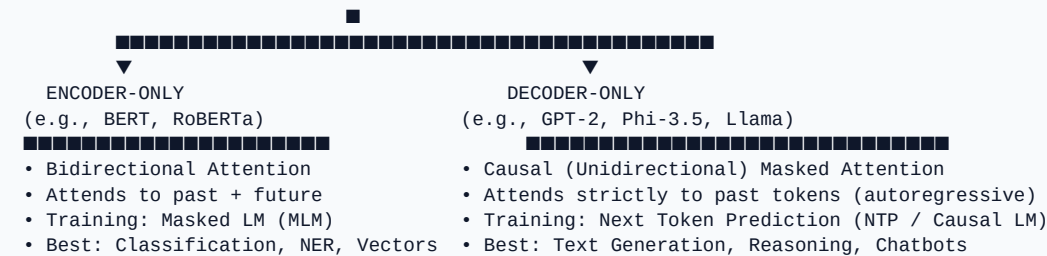
The Complete Master Reference Guide: From Hugging Face Fundamentals to High-Performance Local Production with Ollama

Target Models: BERT, GPT-2, Microsoft Phi-3.5-mini-instruct, Llama • **Key Stack:** transformers, datasets, peft, trl, bitsandbytes, llama.cpp, ollama

1. Core Architectural Foundations: Encoders vs. Decoders

Transformers fall into two primary structural families based on attention mechanics and objective formulation:

TRANSFORMER TAXONOMY

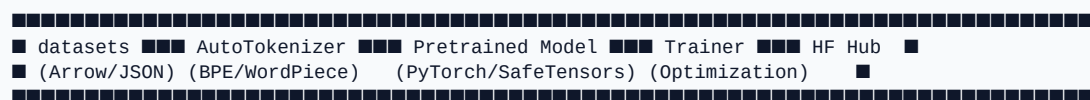


Mathematical Attention Intuition:

- **Encoders (BERT):** Self-attention is calculated over all N tokens simultaneously: $\text{Attention}(Q,K,V) = \frac{\text{softmax}((Q K^T) / \sqrt{d_k}) V}{\sqrt{d_k}}$. Every token directly attends to every other token.
- **Decoders (GPT/Phi):** Enforces a lower-triangular causal attention mask ($-\infty$ in the upper triangle before softmax). Token t cannot look ahead to token $t+1$, ensuring causal autoregressive generation.

2. The Hugging Face Ecosystem & Core APIs

The Hugging Face pipeline forms a standardized workflow from data ingest to tokenization, model loading, and evaluation:



Core Tokenizer Outputs:

- `input_ids`: Integer IDs mapping tokens to vocab indices (e.g. `[101, 2054, 102]`).
- `attention_mask`: 1 for authentic tokens, 0 for padded filler tokens.
- `apply_chat_template()`: Uses Jinja2 templates stored in `tokenizer_config.json` to apply precise conversational role delimiters (e.g., `<|user|>`, `<|assistant|>`).

3. Encoder Fine-Tuning: BERT for Sequence Classification

For classification tasks, BERT extracts the pooled `[CLS]` token embedding (768-dim vector) and passes it to a linear feedforward head with cross-entropy loss over target labels:

```

import torch
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          DataCollatorWithPadding, Trainer, TrainingArguments)
from sklearn.metrics import accuracy_score, f1_score

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=5)

# Dynamic mini-batch padding (pads each batch only to its longest item)
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(axis=-1)
    return {'acc': accuracy_score(labels, preds), 'f1': f1_score(labels, preds, average='macro')}

training_args = TrainingArguments(
    output_dir='./bert_classifier',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,           # Standard fine-tuning learning rate
    weight_decay=0.01,           # L2 regularization
    warmup_ratio=0.1,            # Linear warmup over first 10% steps
    eval_strategy='epoch',
    save_strategy='epoch',
    fp16=torch.cuda.is_available(),
    report_to='none'
)

```

4. Decoder Instruction Fine-Tuning: GPT-2 & Label Loss Masking

In instruction tuning, the model must **only be penalized for predicting the assistant's response**, not the user prompt. We mask prompt token labels with -100:

```

Tokens: [ <|user|>, What, is, 2+2?, <|assistant|>, 4, <|end|> ]
Labels: [ -100, -100, -100, -100, -100, 4, <|end|> ]
          ■■■■■■■■■■ MASKED (-100) ■■■■■■■■■■ ■■■■ LOSS COMPUTED ■■■■

```

Why -100? PyTorch's `nn.CrossEntropyLoss(ignore_index=-100)` ignores targets set to -100 by default. Zero gradient is backpropagated through prompt tokens.

Adding Special Tokens & Embedding Resizing:

```

# 1. Define and add custom tokens
USER_TOKEN, ASSISTANT_TOKEN = '<|user|>', '<|assistant|>'
tokenizer.add_special_tokens({'pad_token': tokenizer.eos_token, 'additional_special_tokens': [USER_TOKEN, ASSISTANT_TOKEN]})

# 2. Critical: resize model embedding matrix to fit new vocab size!
model.resize_token_embeddings(len(tokenizer))

# 3. Target Label Masking Function
def prepare_input(example, max_length=512):
    encoding = tokenizer(example['text'], truncation=True, max_length=max_length, padding='max_length')
    labels = [-100] * len(encoding['input_ids'])
    assistant_id = tokenizer.convert_tokens_to_ids(ASSISTANT_TOKEN)
    if assistant_id in encoding['input_ids']:
        pos = encoding['input_ids'].index(assistant_id)
        for i in range(pos + 1, len(encoding['input_ids'])):
            if encoding['attention_mask'][i] == 1:
                labels[i] = encoding['input_ids'][i]
    return {'input_ids': encoding['input_ids'], 'attention_mask': encoding['attention_mask'], 'labels': labels}

```

5. Modern Chat LLMs: Phi-3.5-mini & Chat Templates

Modern instruction-tuned models (e.g. Microsoft Phi-3.5-mini, Llama-3) use Jinja templates and dynamic sequence-to-sequence collation:

```
messages = [
    {'role': 'system', 'content': 'You are an agricultural expert.'},
    {'role': 'user', 'content': 'How do I identify and treat powdery mildew?'}
]
prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
# Output: '<|system|>\nYou are...<|end|>\n<|user|>\nHow do...<|end|>\n<|assistant|>\n'

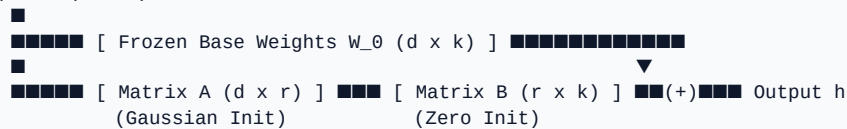
# Dynamic Seq2Seq batch padding (automatically converts pad positions in labels to -100)
from transformers import DataCollatorForSeq2Seq
collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model, padding='longest')
```

6. Parameter-Efficient Fine-Tuning (PEFT) & LoRA Mathematics

Full fine-tuning updates the entire weight matrix $W_{\text{O}} \in \mathbb{R}^{d \times k}$. **LoRA (Low-Rank Adaptation)** freezes W_{O} and decomposes updates into two low-rank matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ ($r \ll \min(d, k)$):

$$h = W_0 x + (\alpha / r) * (B \cdot A) x$$

Input x (dim d)



Parameter Reduction Example: For $d=4096$, $r=8$:

- Full weight matrix: $4096 \times 4096 = 16,777,216$ params.
- LoRA matrices: $(4096 \times 8) + (8 \times 4096) = 65,536$ params (**99.6% parameter reduction!**).

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=8, # Low-rank dimension
    lora_alpha=16, # Scaling multiplier (scale = alpha / r = 2.0)
    lora_dropout=0.05, # Regularization dropout for LoRA layers
    bias='none',
    task_type='CAUSAL_LM',
    target_modules=['qkv_proj', 'o_proj'] # Targeted attention projection layers
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# Trainable: ~3.0M / 3.82B (0.079% of model weights)
```

7. Quantization & QLoRA with SFTTrainer

Quantization maps 32-bit floating point weights into lower precision discrete bins ($X_{\text{quant}} = \text{round}(X / \text{scale}) + \text{zero_point}$):

Precision	Bits / Weight	7B Model VRAM	Quality Retention
FP32 (Standard)	32 bits	~28.0 GB	100% (Baseline)
FP16 / BF16	16 bits	~14.0 GB	99.9%
INT8 (8-bit)	8 bits	~7.0 GB	99.5%
NF4 (4-bit QLoRA)	4 bits	~3.8 GB	98.5%

```
from transformers import BitsAndBytesConfig, AutoModelForCausalLM
from trl import SFTConfig, SFTTrainer

# 1. Load base model in 8-bit quantization
bnb_config = BitsAndBytesConfig(load_in_8bit=True, llm_int8_threshold=6.0)
model = AutoModelForCausalLM.from_pretrained('gpt2-xl', quantization_config=bnb_config, device_map='auto')

# 2. Configure Supervised Fine-Tuning with SFTTrainer
sft_config = SFTConfig(
    output_dir='./qlora_output',
    dataset_text_field='formatted_text',
    max_seq_length=512,
    packing=True, # Efficiently packs short sequences into 512 chunks
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    logging_steps=10
)
trainer = SFTTrainer(model=model, args=sft_config, train_dataset=dataset, peft_config=lora_config, processing_class=tokenizer)
trainer.train()
```

8. Full Deployment Lifecycle: Merging, GGUF & Ollama

Deploying fine-tuned models locally involves fusing LoRA adapters, binary GGUF quantization, and Ollama containerization:



Step-by-Step Production Commands:

```
# Step 1: Merge adapters into standalone model
merged_model = model.merge_and_unload()
merged_model.save_pretrained('./phi-3.5-merged', safe_serialization=True)
tokenizer.save_pretrained('./phi-3.5-merged')

# Step 2: Convert to GGUF using llama.cpp
# python3 llama.cpp/convert_hf_to_gguf.py ./phi-3.5-merged --outfile phi-3.5-f16.gguf --outtype f16

# Step 3: Modelfile definition
# FROM ./phi-3.5-f16.gguf
# TEMPLATE """"{{ if .System }}<|system|>\n{{ .System }}<|end|>\n{{ end }}
# {{ if .Prompt }}<|user|>\n{{ .Prompt }}<|end|>\n{{ end }}<|assistant|>\n""""
# PARAMETER temperature 0.7
# PARAMETER stop "<|end|>"

# Step 4: Register and run in Ollama
# ollama create my-phi3 -f Modelfile
# ollama run my-phi3
```

9. Master API Cheat Sheet & Function Glossary

Quick reference table covering all critical functions, classes, and parameters encountered throughout the course:

Function / Class	Module	Exact Purpose & Critical Parameters
AutoTokenizer.from_pretrained	transformers	Loads tokenizer vocabulary & configuration from model checkpoint.
apply_chat_template	transformers	Formats role messages (system/user/assistant) into model's Jinja template.
resize_token_embeddings	transformers	Expands embedding matrix when new special tokens are registered.
get_memory_footprint	transformers	Returns active memory occupied by model parameters in bytes.

<code>DataCollatorWithPadding</code>	transformers	Dynamically pads mini-batches to max batch sequence length.
<code>DataCollatorForSeq2Seq</code>	transformers	Dynamically pads inputs and fills label pad positions with -100.
<code>LoraConfig</code>	peft	Defines rank (\$r\$), alpha (\$\alpha\$), dropout, and target layers (e.g. qkv_proj).
<code>get_peft_model</code>	peft	Wraps PyTorch base model with trainable low-rank LoRA matrices.
<code>merge_and_unload</code>	peft	Fuses LoRA adapter weights permanently into base model weights.
<code>BitsAndBytesConfig</code>	transformers	Enables 8-bit (load_in_8bit) or 4-bit (NF4) quantized model loading.
<code>SFTTrainer</code>	trl	High-level Supervised Fine-Tuning trainer supporting sample packing & PEFT.
<code>convert_hf_to_gguf.py</code>	llama.cpp	Converts Hugging Face checkpoints to GGUF binary format for llama.cpp.
<code>ollama create / run</code>	CLI	Builds and executes local LLM containers with hardware acceleration.